**Huddle** Decision-led development for teams coding with AI

# How a team works with Claude Code

Each person works with Claude Code on their own computer. Tasks arrive as Issues on GitHub or GitLab, ready to start, and Huddle keeps their status up to date by itself. This guide explains the cycle, the roles, the rules and how to start on your first day.

> **Our job is no longer typing code. It is making good decisions. The AI writes, tests and asks; the person decides.**

THE CYCLE

## From an idea to merged code, step by step

1

AI MANAGER

### Plans and splits the work

The AI Manager defines what gets built, writes the spec and splits it into tasks that do not step on each other: each one touches different files. When one task needs another first, it says so under «Depends on».

2

AI MANAGER

### Creates the Issue with its prompt ready

With `/huddle-issue`, each task becomes an Issue from the template: the exact prompt to paste into Claude Code, the files that may and may not be touched, and the tests that must pass. Huddle assigns it to the person with the fewest open Issues.

3

DEVELOPER

### Starts it with one command

`/huddle-play N` checks that every dependency is merged, creates the branch in a folder of its own, marks the Issue `in progress` and runs its prompt. There is no need to explain the project: `CLAUDE.md` already gives the session the context. Run the same `/huddle-play N` again once the AI Manager has answered a review or a decision: the session sees the Issue is already started and continues it.

4

DEVELOPER · THIS IS WHERE THE VALUE IS

### Answers the AI's questions and decides

The session asks what the Issue does not settle: what to do in an unusual case, what text the user sees, what to prioritize. Each answer is a decision, and the quality of the result depends on it. If the work cannot go on without the AI Manager, the session asks a «❓ Decision for the AI Manager» on the Issue and stops.

5

CLAUDE CODE

### Writes the test, the code and opens the PR

First the failing test, then the code that makes it pass. It opens a pull request (a merge request on GitLab) with the tests it ran and their real results, and closes with a message that starts with **«✅ Issue #N finished»**. Then it keeps waiting for the review in the background.

6

THE PLATFORM

### Automatic tests and labels

The project's CI runs its tests. Huddle's labels job moves each Issue through `ready`, `assigned`, `in progress`, `in review` and `blocked`, from its «Depends on» and its PR. Nobody moves those labels by hand.

7

AI MANAGER

### Reviews: asks for changes or merges

With `/huddle-review`, the AI Manager either comments a request with a «Prompt for Claude Code» block, which the waiting session takes by itself (marking it 👀), or approves and merges. A merged PR closes the Issue and unblocks the ones that depend on it. Anything that touches production or secrets is done only by the AI Manager. To have its session check by itself what waits, the AI Manager runs `/huddle-watch` (see «For the AI Manager»).

ROLES

## Who does what

### AI Manager (a single person)

*A person, not the AI: leads the work with the help of their own Claude Code session. Their login is* `ai_manager` *in* `huddle.json`.

- Decides what gets built and in what order.
- Writes the specs and the Issues with their prompt.
- Answers every «❓ Decision for the AI Manager».
- Reviews and merges PRs. The only one who merges into the default branch.
- The only one who touches production, settings and secrets.
- Adds and removes people from the team.

### Developer

*Listed under* `assignees` *in* `huddle.json`.

- Takes an assigned Issue and works on its own branch.
- Answers the session's questions with judgment.
- Reviews what the AI did before the PR is opened.
- If something does not fit the Issue, asks the AI Manager instead of improvising.
- Never merges, never touches production.

THE ISSUE

## What each task brings

The Issue template makes every task arrive complete, so anyone's session starts the same way, no matter who takes it.

| | |
|---|---|
| **Goal** | What we want to achieve and why, in a few lines. |
| **Depends on** | The Issues that must be merged first, one per line, or «None». Huddle keeps the Issue `blocked` until they are. |
| **Branch** | The name of the branch, created from an up-to-date default branch. |
| **Prompt for Claude Code** | The text the session runs, with the steps, the tests and how to open the PR. |
| **Allowed files** | The only things the session may touch. Anything else: ask first. |
| **Contract or spec** | The document that rules over the code. |
| **Tests that must pass** | What must pass before the PR, reported with real counts. |

LABELS

## Where each Issue stands, at a glance

Huddle's labels job updates these on every change (on GitLab, by a schedule every 10 minutes). Filter the Issue list by a label to see, for example, everything waiting for review.

| | |
|---|---|
| `ready` | Its dependencies are merged and nobody has it yet. |
| `assigned` | Someone has it but has not started. |
| `in progress` | Its session started it, or took a request from the AI Manager. |
| `in review` | Its PR is open and waiting for the AI Manager. |

| | |
|---|---|
| `blocked` | It waits for an Issue under «Depends on» to be merged. Closing that Issue by hand does not count. |
| `needs decision` | The session asked the AI Manager a question and stopped. |
| `changes requested from AI Manager` | A review request is waiting for the session to take it. |

**The stop labels.** `needs decision` and `changes requested from AI Manager` mean the work waits for someone else, so they stand alone: `in progress` and `in review` are taken off while one of them is on, and come back by themselves once the request is taken or the question is answered. Nobody removes them by hand.

PROGRESS

## How far each area has come

Each area of the project (for example «Content», «Social networks», «Ads») is a milestone, and each Issue belongs to one. The AI Manager writes in each milestone's description how many Issues are still missing, on a line `Huddle remaining: N`. Huddle then shows, per area and in total, the merged Issues over all the work the area will need:

- in a pinned «Huddle progress» Issue that the labels job keeps up to date;
- with `huddle progress` (ask Claude Code «run huddle progress»);
- in one line when a session opens, for example `Content 76% · Social networks 55% · Ads 0% · Total 58%`.

Issues closed by hand or «not planned» do not count, and Issues with no milestone stay out of the percentage.

**Not sure which areas to use?** The AI Manager runs `/huddle-areas`. Claude reads the README, the specs and the Issues, proposes the areas with the reason for each (as many as the project needs), puts the Issues that have no area in one, and suggests how many Issues are still missing in each. It explains first and changes nothing until the AI Manager says yes. It can be run again later to check that the areas are up to date.

WHY

## Why working this way is better

### Before

- Most of the time went into writing code by hand.
- Each person explained the project in their own way.
- Tasks collided in the same files.
- Mistakes showed up late, in review or in production.

### Now

- The person's time goes into deciding and reviewing.
- Every session reads the same context and the same rules.
- Each task has its own files: two people do not collide.
- Tests and review catch mistakes before merging.

### The decision is the work

The AI is fast at writing and testing, but it does not know the business, the users or the priorities. When it asks, the person brings what the AI does not know. A good answer saves hours; a rushed answer is paid for later.

### A ready prompt removes variation

If each person writes their own request, each session understands something different. With the prompt written by the AI Manager, they all start with the same goal, the same limits and the same tests.

### Clear limits, fewer collisions

Allowed and forbidden files let several people move forward at once without stepping on each other, and tell the AI where not to go.

### Production in a single pair of hands

Nobody on the team needs keys or access to production. A mistake in one session cannot touch what customers use.

A FREQUENT QUESTION

## Why doesn't a developer open 10 sessions to take 10 Issues at once?

It sounds faster, but in practice a developer with many open sessions gets less done and decides worse.

### The computer becomes the bottleneck

Each session runs tests and builds. With many at once, the machine slows down and they all move more slowly than a few would.

### Attention does not multiply

Each session asks questions. With ten open, the questions pile up, you lose the thread and start answering fast just to get through. That is when the decision stops being good, and the decision is exactly what the person brings.

### Every session has to be reviewed

Ten half-reviewed PRs are worth less than two done well, and they fill up the AI Manager's review queue.

**Our recommendation:** one or two active sessions per person. Huddle assigns at most a few open Issues per person for the same reason. More speed comes from adding developers, not from adding windows.

RULES

## What is not negotiable

**Never push to the default branch.** One task is one Issue, one branch and one PR. Only the AI Manager merges.

**Only the allowed files.** If another one is needed, ask first.

**Test first.** Write the failing test, then the code. The PR says which tests were run and with what result.

**No production and no secrets.** If a permission or a key is missing, stop and tell. Do not look for another way.

**Nothing is dropped silently.** Everything the Issue asks for ends up done, or listed under «What was NOT done» with the reason.

**Close the task clearly.** Every session ends with «✅ Issue #N finished» or «⛔ Issue #N stopped», the PR link, the test counts and your next step. If you do not see that message, the session is not over.

**Chat in your language; the repository is in English.** Commits, PRs, Issues, code comments and new documents are in English.

FOR THE AI MANAGER

## Setting up, adding people, watching and updating

### 1. Put Huddle in the repository

On a branch, run `/huddle-setup`. It asks the platform, the project, the AI Manager and the team, then writes `huddle.json`, the Issue and PR templates, the labels job, the Huddle block of `CLAUDE.md` and `.claude/settings.json`. Merge it with a PR, then ask Claude Code «run huddle doctor». If its `login` line says «gh is not installed or not on the PATH» (or `glab`), the tool is missing from that terminal: it gives the install command, and on Windows you open a new terminal afterwards. «Is not logged in» is a different message, with `gh auth login` or `glab auth login` as the fix.

**On GitLab** two more things, only by you: a project access token with the `api` scope stored as the masked and protected CI variable `HUDDLE_TOKEN`, and a pipeline schedule `*/10 * * * *` on the default branch. Never paste the token in a chat or a file.

### 2. Add someone to the team

Invite them to the repository (GitHub: *Settings, Collaborators*; GitLab: *Manage, Members*, role Developer). Then run `/huddle-team`: it checks that the login can be assigned, adds it to `assignees` in `huddle.json` through a PR, and gives you the message to send them with the install steps. `huddle team` lists the team and shows who cannot be assigned yet. Once that PR is merged, Huddle assigns them `ready` Issues.

### 3. Let the session review by itself

Run `/huddle-watch` (also `/huddle-watch status` and `/huddle-watch off`). Every few minutes the AI Manager's session checks what waits and reviews it. Turning it on is the order to merge what it approves; decisions and bug triage stay with the person. `huddle queue` lists what waits (ask Claude Code «run huddle queue»). The watch lives only in that session: closing Claude Code ends it.

### 4. Update a project to a new release

Install and update are per project, so do this in each project where Huddle is used. In a terminal, from the project's folder, run `claude plugin marketplace update huddle` and then `claude plugin update huddle@huddle --scope project`, and open a new session. Then, on a branch, run `/huddle-update`: it refreshes the templates and the `CLAUDE.md` block and asks only the questions a new release added. A project can be on an older version than another; `claude plugin list` shows each one. In the VS Code extension, which has no `claude` command in a terminal, type `/plugins` and use the same dialog to install (**Install for this project**) or update it.

A newer release is announced in the chat by `/huddle-play` and `/huddle-update`, at most once a day. `huddle notice` says the same on demand.

GETTING STARTED

# Your first day, step by step

You do this once. It takes 20 to 40 minutes, mostly waiting for downloads. When a step depends on your computer, there is one block for Windows and another for Mac. Your project may add its own steps (its libraries, its test database): they are in the project's `README` and step 7 points you there.

**Copy each command from its box, whole and on a single line.** If it pastes split in two lines, join it before pressing Enter.

## 1. Choose how you will use Claude Code

Claude Code works the same in all its versions: the same rules, the same `CLAUDE.md` and the same Huddle plugin. Pick the one you find most comfortable; you can switch later.

DESKTOP APP · THE EASIEST

For people who are starting out. You see the changes visually and can have several sessions side by side. Requires a paid Claude plan.

**Install:** follow the Windows or Mac download links at `code.claude.com/docs/en/desktop-quickstart`, sign in and open the **Code** tab.

VS CODE EXTENSION

For people who want to read and edit the code in an editor while working with Claude Code.

**Install:** VS Code from `code.visualstudio.com`; then open the extensions panel (`Ctrl+Shift+X` or `Cmd+Shift+X`), search for **Claude Code** (by Anthropic) and install it. The extension has no terminal command to type for Huddle: you install it with `/plugins` in step 4.

TERMINAL (CLI)

For people who already use the command line. It is also the one that shows Huddle's notice the moment a session opens.

**Install:** on Windows, in PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

On Mac, in Terminal:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then type `claude` inside the project folder.

### 2. Install Git and your platform's command line

Git keeps the code's history. Huddle talks to your platform through its official command line: `gh` for GitHub, `glab` for GitLab. Install the one your project uses.

WINDOWS

1. **Git:** download «Git for Windows» from `git-scm.com` and install it with the default options. It includes **Git Bash**.
2. **GitHub:** open PowerShell and type:

   ```
   winget install --id GitHub.cli
   ```

3. **GitLab:**

   ```
   winget install --id GLab.GLab
   ```

4. Close the terminal and open a new one, so it finds the new programs. In VS Code, close every window and open it again.

MAC

1. **Git:** open Terminal and type (two hyphens before `install`):

   ```
   xcode-select --install
   ```

2. **Homebrew** (the Mac's program installer): follow the one-line instruction at `brew.sh`.

3. **GitHub:**

```
brew install gh
```

**GitLab:**

```
brew install glab
```

**Check:** in a new terminal, `gh --version` (or `glab --version`) answers with a version number.

### 3. Connect your accounts

**The repository:** accept the invitation that arrives by email. Until you accept it, Huddle cannot assign you Issues.

**GitHub:** type `gh auth login` and choose GitHub.com, HTTPS and «Login with a web browser».

**GitLab:** type `glab auth login`, choose gitlab.com (or your company's server) and sign in with the browser. Then let Git use `glab` for gitlab.com, so a Claude Code session can push without opening a browser:

```
git config --global --add credential.https://gitlab.com.helper "!glab auth git-credential"
```

**Claude Code:** open it and sign in with the Claude account the AI Manager tells you.

### 4. Install the Huddle plugin

Huddle comes as a Claude Code plugin. Each person installs it once on their own computer; committing it to the repository does not install it for you. In a terminal (if you use only the VS Code extension, skip these two commands and follow the next paragraph):

```
claude plugin marketplace add SrProfessional/Huddle
```

```
claude plugin install huddle@huddle
```

**VS Code extension only:** type `/plugins`, add the marketplace `SrProfessional/Huddle` in the **Marketplaces** tab, install `huddle` from the **Plugins** tab with **Install for this project** (not «Install for you»). Each person does this once per clone.

Then open a new Claude Code session. In the CLI you can also type `/plugin marketplace add SrProfessional/Huddle` and `/plugin install huddle@huddle` instead.

**Check:** `claude plugin list` shows `huddle@huddle` enabled (in the VS Code extension, `/plugins` shows it in the **Plugins** tab). To update it later, in VS Code use the same `/plugins` dialog; otherwise run `claude plugin marketplace update huddle` and `claude plugin update huddle@huddle --scope project` from the project's folder, open a new session, and on a branch run `/huddle-update`: it asks only the questions a new release added.

### 5. Set your language (a single message)

Issues come in English, so Claude Code would answer in English unless you set your language. Open a session and send this message, replacing `YOUR_LANGUAGE` with your language written in English (for example `spanish` or `portuguese`):

```
Add "language": "YOUR_LANGUAGE" to my personal Claude Code settings file,
~/.claude/settings.json. Keep everything else in that file as it is.
```

It asks for permission to edit the file: accept it. From then on it answers in your language in every session. Commits, PRs and comments stay in English.

### 6. Download the project and open it

In a terminal, go to the folder where you keep your projects and clone it (the AI Manager gives you its name):

```
gh repo clone OWNER/PROJECT
```

On GitLab: `glab repo clone OWNER/PROJECT`. Then open that folder in the Claude Code you chose. When it asks whether you trust the folder, say yes.

### 7. Do your project's own setup

Every project has its own libraries and tests. Open its `README` (and its `CLAUDE.md`) and follow the setup section, or ask Claude Code: «read the README and install what the project needs, then run the tests and tell me the results».

### 8. Check that Huddle sees everything

Ask Claude Code: «run huddle doctor». Every line must show ✓: `plugin`, `huddle.json`, `login`, `ai_manager` and `labels`. If one shows ✗, its line says what to do (for `login`: «not installed or not on the PATH» means `gh` or `glab` is missing from this terminal, and the line gives the install command; «not logged in» means run `gh auth login` or `glab auth login`); if it is not clear, send it to the AI Manager.

### 9. Take your first Issue

Find the Issue assigned to you (label `assigned`), or type `gh issue list --assignee @me`. In Claude Code type:

```
/huddle-play N
```

with the Issue's number instead of `N`. It checks that the Issue's dependencies are merged, creates your branch in a folder of its own, marks the Issue `in progress` and starts the Issue's prompt. Answer its questions calmly: that is where your work is.

### 10. When the AI Manager asks for changes

You do not need to watch GitHub. After its closing message the session keeps waiting for the review. When the AI Manager leaves a request, the session tells you and asks whether to continue: say yes. You can also type `/huddle-play N` again yourself, with the Issue's number: it picks up the AI Manager's answer. If you had closed Claude Code, open a new session in the project: Huddle tells you that a request is waiting (in the CLI you see the line at once; in the desktop app and the VS Code panel Claude says it after your first message).

Huddle · decision-led development for teams coding with AI · github.com/SrProfessional/Huddle. The current rules of your project are in its `CLAUDE.md`.